RICE UNIVERSITY

Processes and Threads

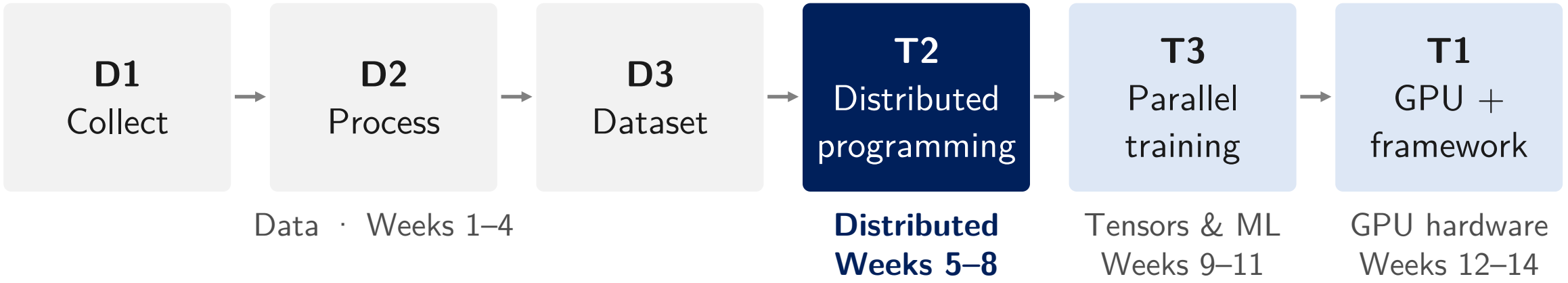
Using many cores: what runs, what is shared, and how much faster it gets

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 5, Lecture 1

Where we are in the pipeline



In our pipeline

Today: many workers on **one machine** — processes, threads, and what they share. Spark (Weeks 3–4) did this for you; now we see how.

Last time (Week 4): tokenization at scale with Spark. D2 due Wednesday. Next: networking (Week 6).

Today's plan

1

Processes and threads

What does the operating system give a running program?

2

Races, locks, and deadlock

What goes wrong when threads share data?

3

Python, and the limits of speedup

How much faster can p workers make a job?

Build

Threads, processes, and two bugs

Work in pairs on a laptop. Predict first, then run.

Why do we need more than one core?

The job: normalize and count words in all of English Wikipedia
$\approx$ **7 million** articles, $\approx$ **10 ms** of CPU time each (assumed)

On one core, how long does that take?

One core
$\approx$ **19 hours**

64 cores, perfectly split
$\approx$ **18 minutes**

Key idea

Since about 2005 single cores have **not** got much faster; chips add **more cores**. A program is fast only if it uses many at once.

Source: Wikipedia, "Size of Wikipedia"; 10 ms per article and 64 cores assumed; Sutter, Dr. Dobb's Journal, 2005.

Processes and threads, in plain words

Process A

its own memory: code, variables

thread 1 | thread 2 | thread 3

Process B

its own memory: code, variables

thread 1

Operating system scheduler: which thread runs on which core, and when

core 0 | core 1 | core 2 | core 3

Definition

Process: a running program with its **own memory**. **Thread**: one sequence of instructions inside a process; threads **share** its memory.

Where processes and threads show up

- **Spark** (Weeks 3–4): each executor is a **process**; it runs tasks on **threads**.
- **PyTorch**: `DataLoader(num_workers=4)` loads batches in **4 processes**.
- **T2** (Weeks 7–8): one **process per rank** with mpi4py and torch.distributed.

The price

Processes: **isolated**, so they must **copy** data to talk.

Threads: **share** memory, so they can **corrupt** it.

Why do we need processes and threads?

One core is not enough. Using many means choosing between **processes** and **threads**, and controlling what they **share**.

Source: Apache Spark documentation, Cluster Mode Overview; PyTorch documentation, torch.utils.data.

Learning goals for today

- **Explain** process versus thread: address spaces, scheduling, concurrency versus parallelism.
- **Find** a race condition and a deadlock, and fix them with locks and a lock order.
- **Derive** Amdahl's law and use it to predict the speedup of a job.
- **Choose** threads or processes in Python, knowing what the **GIL** does.

A puzzle to start

One call of a pure-Python function takes $\approx$ **0.6 s** on one core.
Run **4 calls** at once on a laptop with **10 cores**.

4 threads, or 4 processes: how long does each take?

Answer

Threads $\approx$ 2.3 s — no faster than one after another. **Processes $\approx$ 0.7 s.** Part 3 explains why.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Part 1

Processes and threads

What does the operating system give a running program?

1 · Processes and threads

2 · Races, locks, and deadlock

3 · Python, and the limits of speedup

Build · Threads, processes, and two bugs

A program becomes a process

program
job.py on disk
(a file)

run →

process (PID 4711)

its own **memory**: code, variables, stack

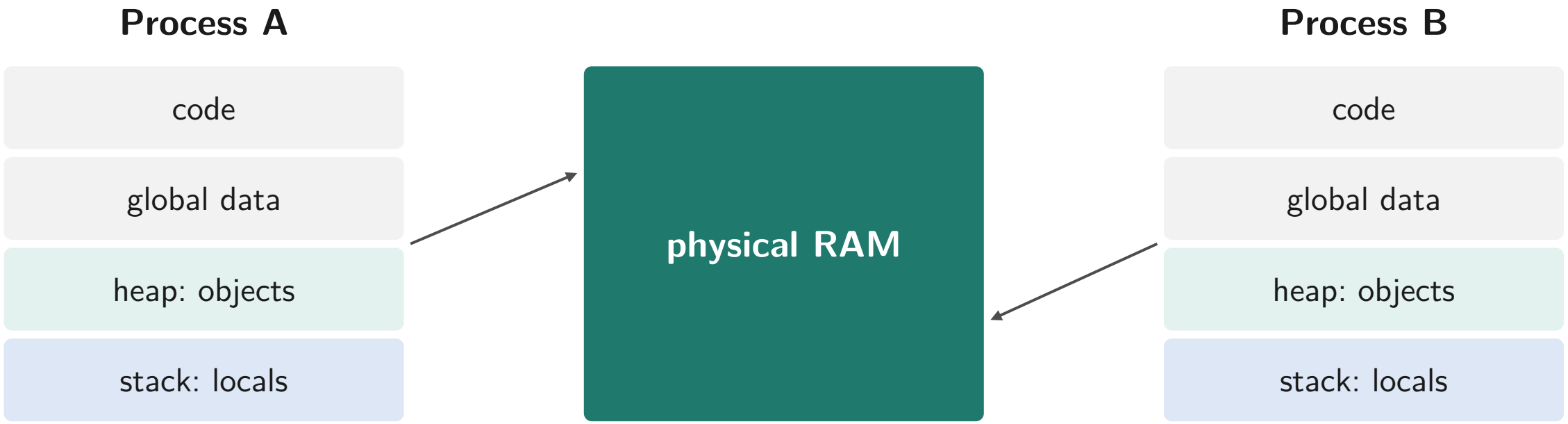
one **thread** to start: where it is in the code

resources: open files, network connections

Key idea

A **process** is a running program. The OS gives it memory, a first thread, and resources, and takes them back when it ends.

Each process has its own address space



the OS maps each process to different parts of RAM

Memory

A process **cannot see** another process's memory: isolation is safe, but sharing data means **copying** it.

Processes do not share variables

```
from multiprocessing import Process
count = 0
def work():
    global count; count += 1      # child's copy
if __name__ == "__main__":
    ps = [Process(target=work) for _ in range(4)]
    for p in ps: p.start(); p.join()
    print(count)
```

What does the parent print?

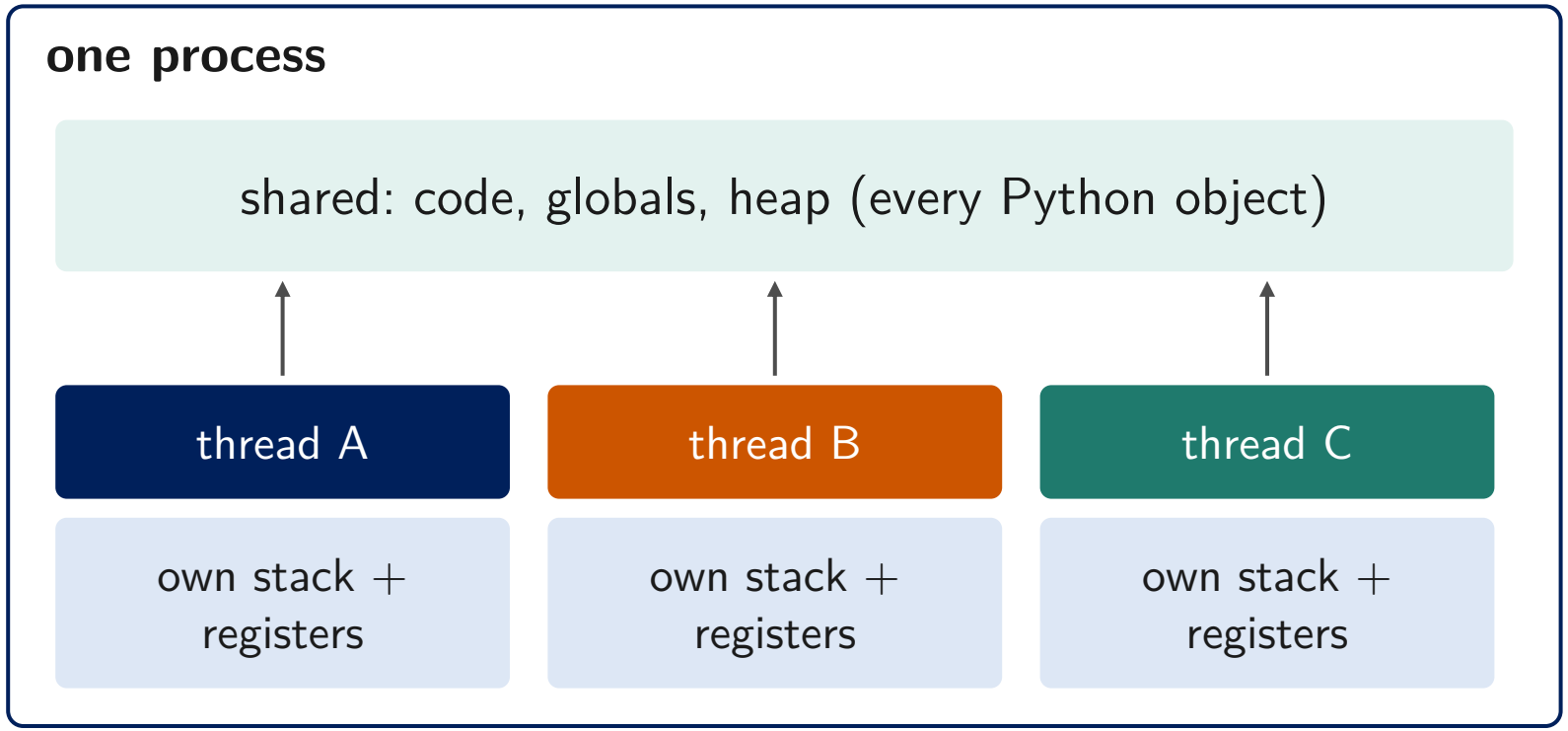
0

Each child changed **its own copy** of count.

Memory

To get results out of a process you must **send** them (a queue, a pipe, a return value): processes communicate by **copying**.

Threads share one address space



- **No copying**: every thread sees every object.
- **Cheap** to start: no new memory.
- **Danger**: two threads can change the same object at once.

Key idea

Threads share the heap and own only their stack: **fast** cooperation, **unsafe** without care (Part 2).

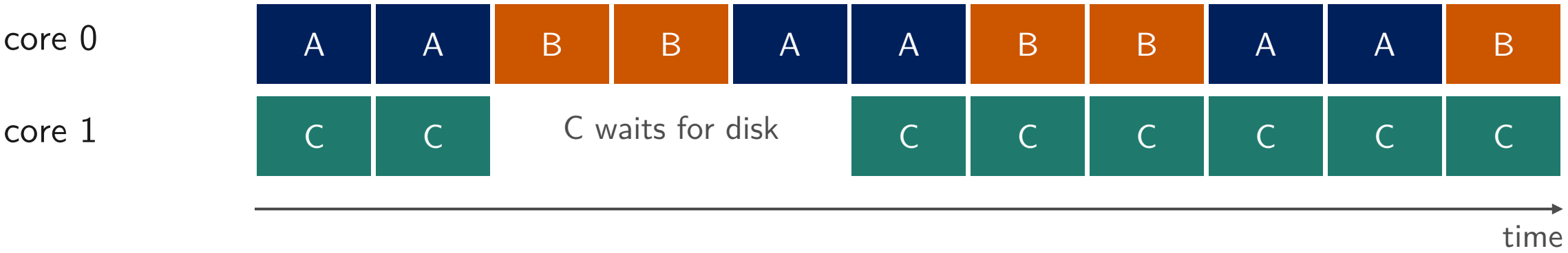
Process or thread?

	Process	Thread
Memory	its own address space	shares its process's
Talking to others	copy data (queue, pipe, network)	read and write shared objects
A bug or crash	stays inside the process	can corrupt the whole process

Key idea

Processes: isolation, paid for in copying. **Threads**: sharing, paid for in bugs. Cluster workers (Weeks 6–8) are processes.

The scheduler shares the cores



- **running**: on a core now.
- **ready**: could run; waiting for a core.
- **blocked**: waiting for disk, network, or a lock; needs no core.

Watch out

The **OS** picks who runs when, so the interleaving of threads **changes from run to run**.

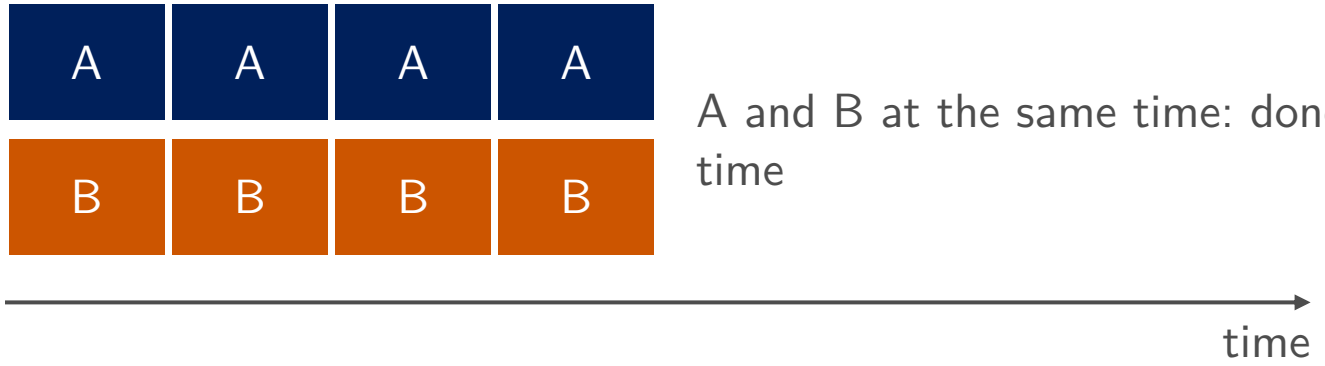
Concurrency is not parallelism

Concurrency: 1 core



A and B take turns

Parallelism: 2 cores



A and B at the same time: done in half the time

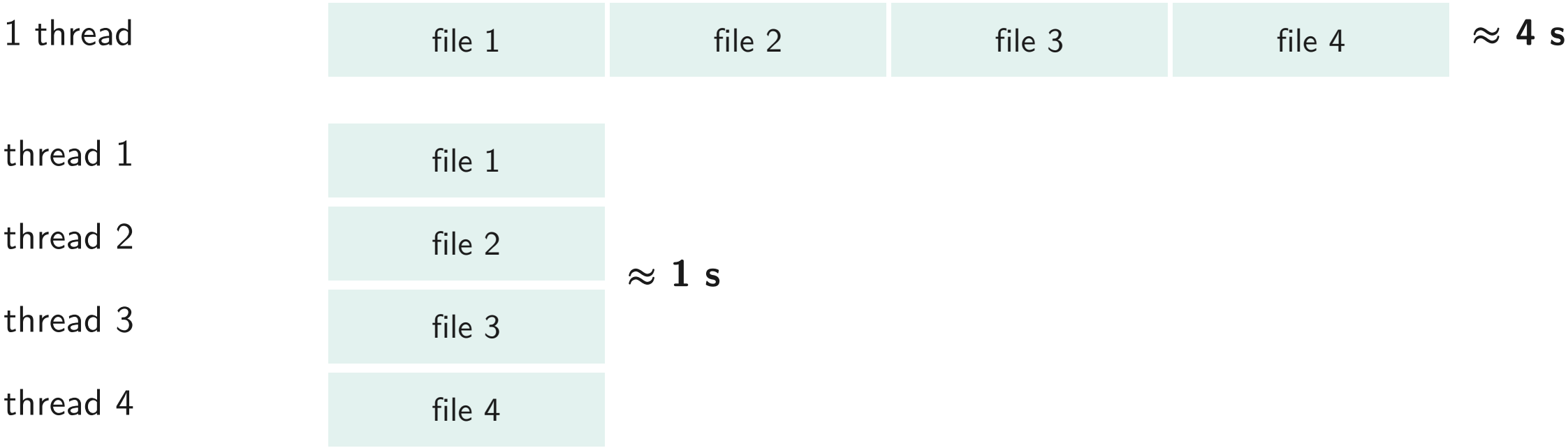
concurrency: tasks overlap in time (they may take turns)

parallelism: tasks run at the same instant, on different cores

Key idea

Only **parallelism** makes CPU-bound work finish sooner. It needs **more than one core**.

Concurrency pays off when threads wait



teal: waiting for the network (a waiting thread needs no core)

Key idea

Waiting (disk, network) **overlaps** across threads even on one core. Computing does not.

Source: time.sleep(1) as the network wait, measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Check yourself

Two minutes with a neighbour.

- What does a thread share with the other threads of its process? What does it own?
- Two threads on **one** core: concurrency, parallelism, or both?
- A script downloads 100 files and spends 95% of its time waiting. Would threads help, even on one core?

Part 2

Races, locks, and deadlock

What goes wrong when threads share data?

1 · Processes and threads

2 · Races, locks, and deadlock

3 · Python, and the limits of speedup

Build · Threads, processes, and two bugs

A race: count += 1 is three steps

thread 1	thread 2	count in memory
read count → 5		5
	read count → 5	5
add 1, write 6		6
	add 1, write 6	6 (should be 7)

Watch out

A **race condition**: the result depends on how the threads interleave. Here one update is **lost**.

A race you can run

```
count = 0
def work(n):
    global count
    for _ in range(n):
        c = count              # read
        time.sleep(0)          # let another thread run
        count = c + 1          # write
```

Expected: **40,000**

Printed: ≈ **10,000**

and a different number every run

Start 4 threads on `work(10_000)`, join them, `print(count)`.

Watch out

Most updates were lost. A race may hide in testing and appear later: **reason** about every shared write; do not trust one lucky run.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

A lock makes the steps indivisible

```
lock = Lock()
def work(n):
    global count
    for _ in range(n):
        with lock:                  # one thread at a time
            c = count
            time.sleep(0)
            count = c + 1
```

Printed: **40,000**, every run

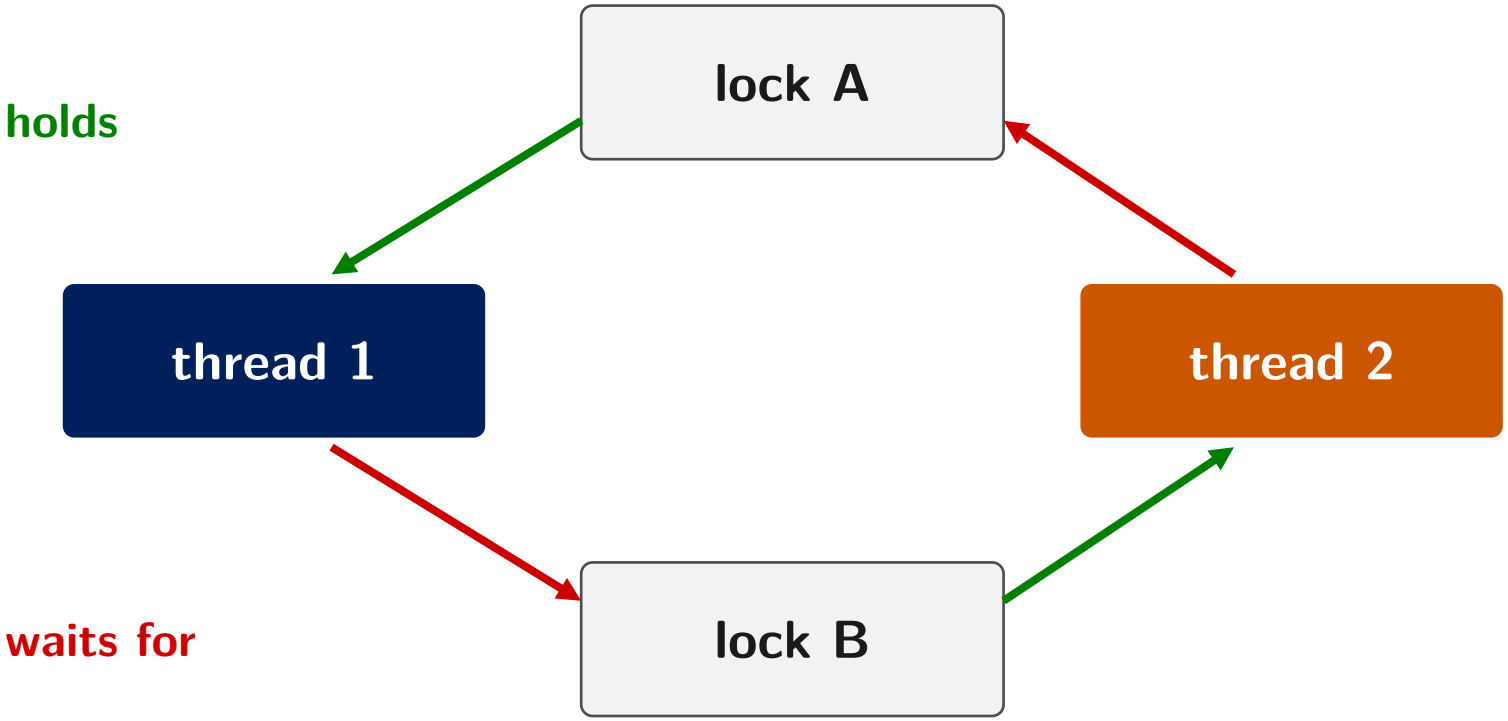
critical section: the code inside `with lock:`

Key idea

A **lock** lets one thread at a time into the critical section. The price: that part now runs **serially** — remember this in Part 3.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Deadlock: each waits for the other



- Thread 1 holds A, waits for B.
- Thread 2 holds B, waits for A.
- A **cycle**: nobody can move, **forever**.

Watch out

A **deadlock** does not crash: the program just **stops**, using no CPU.

A deadlock you can run

```
a, b = Lock(), Lock()
def move(first, second):
    with first:
        time.sleep(0.1)        # hold first ...
        with second:           # ... wait for second
            print("moved")
Thread(target=move, args=(a, b)).start()
Thread(target=move, args=(b, a)).start()
```

Prints **nothing**. Never ends.

Thread 1 takes **a then b**; thread 2 takes **b then a**.

Tip

Detect it with `t.join(timeout=2)` then `t.is_alive()`: a thread still alive after its work should be done is **stuck**.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Avoiding deadlock: one lock order

- Number the locks: A $= 1$, B $= 2$.
- **Every** thread acquires locks in **increasing** order.
- Thread 2 now waits for A first; when thread 1 is done, it gets both.

Why it works

A cycle needs a thread that holds a **higher** lock while waiting for a **lower** one. The order forbids it.

Coming back in Week 7

Week 7: two processes that each **send first, then receive** can deadlock the same way. The fix is again an **order**.

Check yourself

- Why can `count += 1` in two threads lose an update?
- Which code must a lock protect for the race to disappear?
- Thread 1 takes locks A then B; thread 2 takes B then A. What can happen? How do you fix it?

Part 3

Python, and the limits of speedup

How much faster can p workers make a job?

1 · Processes and threads

2 · Races, locks, and deadlock

3 · Python, and the limits of speedup

Build · Threads, processes, and two bugs

Python's global interpreter lock (GIL)



only one thread holds the GIL at a time

- **CPU-bound** Python in threads: one core's worth of work.
- **Waiting** (sleep, files, network) releases the GIL: threads overlap.
- **NumPy / PyTorch** release it inside big operations.

Key idea

One GIL **per process**: processes run Python in parallel; threads in one process do not.

Source: Python 3.14 documentation: threading, multiprocessing, concurrent.futures; PEP 703 and PEP 779.

Threads or processes in Python?

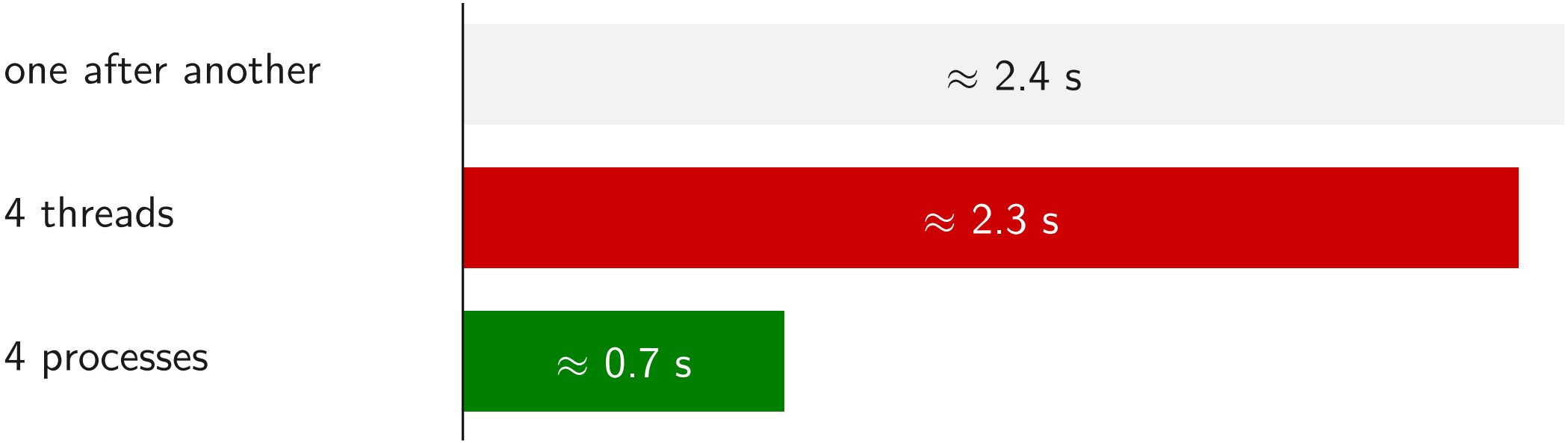
Kind of work	threading	multiprocessing
Pure-Python, CPU-bound	**no speedup** (GIL)	$\approx$ **p**$\times$ **faster**
Waiting on disk or network	**good**: waits overlap	works, but heavier
Large NumPy / PyTorch ops	can help (GIL released)	works; copies data

Key idea

Same interface: `ThreadPoolExecutor` $\leftrightarrow$ `ProcessPoolExecutor`. Processes pay for **copying** arguments and results, so give each one a **big** chunk.

Source: Python 3.14 documentation: threading, multiprocessing, concurrent.futures; PEP 703 and PEP 779.

Back to the puzzle



Key idea

Threads: the **GIL** serializes Python. Processes: $\approx$ **3.4×** — not quite 4×, because starting them does not shrink.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Speedup and efficiency

$$S(p) = T(1) \,/\, T(p)$$
$$E(p) = S(p) \,/\, p$$

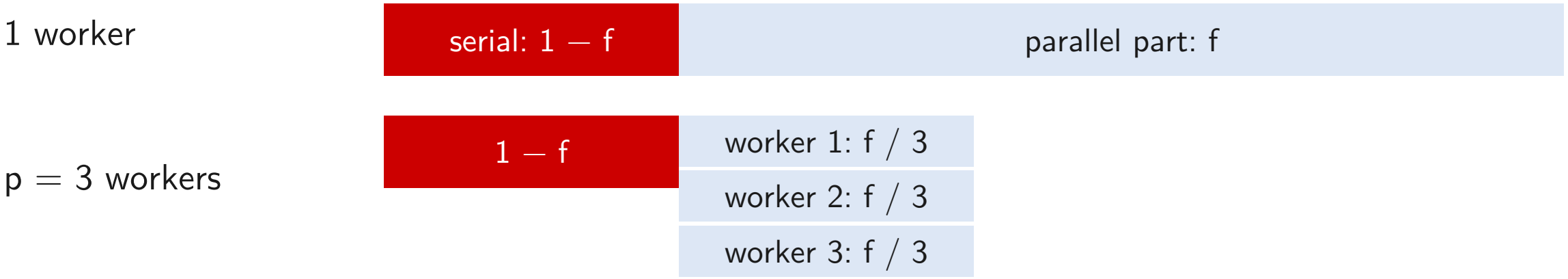
- **T(p)**: time on p workers.
- **speedup S**: how many times faster.
- **efficiency E**: fraction of the ideal p×.

Example: 100 s on 1 worker, 30 s on 4 workers
S = 100 / 30 ≈ **3.3**, E = 3.3 / 4 ≈ **0.83**

Key idea

Ideal speedup is S(p) = p (efficiency 1). Real jobs fall below it — the next slide says why.

Amdahl's law



$$T(p) = (1 - f) \cdot T(1) + f \cdot T(1) / p$$

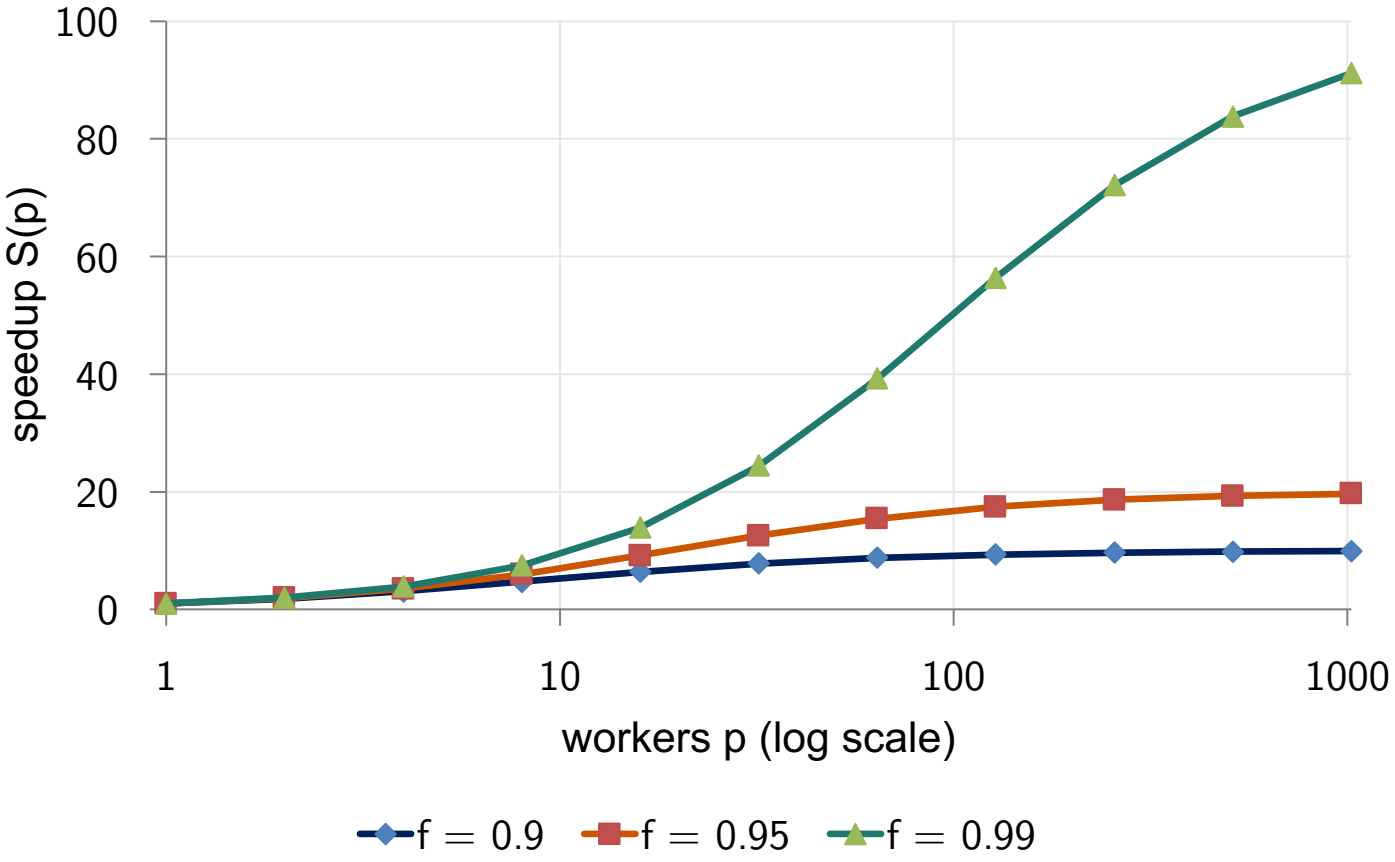
$$S(p) = T(1) / T(p) = 1 / ((1 - f) + f / p)$$

Amdahl's law

Adding workers shrinks **only** the parallel part. The serial part stays the same.

Source: Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

The serial part sets a ceiling



$$p \to \infty:$$

$$S(p) \to 1 \,/\, (1 - f)$$

f = 0.90 → at most 10×

f = 0.95 → at most 20×

f = 0.99 → at most 100×

Watch out

1% serial work caps even **1,000** workers below 100×.

Source: Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

Amdahl on a D2-sized job

One core: 100 min. 5 min of it is serial: the driver plans the job and writes one output file. So f = **0.95** (assumed numbers).

- 8 workers: $5 + 95/8 \approx 16.9$ min $\rightarrow$ **5.9×**
- 64 workers: $5 + 95/64 \approx 6.5$ min $\rightarrow$ **15.4×**
- any number of workers: at most **20×**

In our pipeline

8× the hardware (8 → 64) buys only **2.6×** the speed. Shrink the **serial part** first. Moving data between workers costs extra (Week 6).

Strong and weak scaling

Strong scaling

Same problem, more workers.

Amdahl: $S(p) \leq 1 \,/\, (1 - f)$

Ideal: time falls as $1/p$.

Weak scaling

Problem grows with the workers.

Gustafson: $S = (1 - f) + f \cdot p$

Ideal: time stays flat.

$f = 0.95$, $p = 64$: weak-scaling speedup $\approx$ **61** (versus ≈ 15 for strong scaling)

In our pipeline

D2's scaling study: **strong** = same corpus, more executors; **weak** = corpus grows with the executors.

Source: Gustafson, "Reevaluating Amdahl's Law," Communications of the ACM 31(5), 1988.

Check yourself

- Why do 4 threads not speed up a pure-Python loop, while 4 processes do?
- 90% of a job parallelizes. What is the speedup on 10 workers? The ceiling?
- A job takes 100 s on 1 core and 40 s on 4. Speedup? Efficiency? Name one likely cause.

IN-CLASS BUILD

Threads, processes, and two bugs

Work in pairs on a laptop. Predict first, then run.

1 · Processes and threads

2 · Races, locks, and deadlock

3 · Python, and the limits of speedup

Build · Threads, processes, and two bugs

Exercise 1: threads versus processes

```
def work(n):                          # CPU-bound
    return sum(i * i for i in range(n))

def run(Pool):                        # 4 jobs, 4 workers
    t0 = now()
    with Pool(max_workers=4) as pool:
        list(pool.map(work, [10**7] * 4))
    return now() - t0
```

Call `run(Threads)` and `run(Procs)` under the main guard (processes need it).

DO

1. Predict both times from one call of work.
2. Run. Explain the gap.

STRETCH

Replace the sum with `time.sleep(0.6)`. What changes, and why?

Exercise 2: find the race

```
counts = {}                 # shared by all threads
def count_words(lines):
    for line in lines:
        for w in line.split():
            n = counts.get(w, 0)
            time.sleep(0)       # stands in for real work
            counts[w] = n + 1
```

4 threads, a quarter of the lines each.

DO

1. Which lines race?
2. Fix it with a lock.
3. Fix it with **no** lock.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Exercise 3: find the deadlock

```
def transfer(src, dst, amount):
    with src.lock:
        with dst.lock:
            src.balance -= amount
            dst.balance += amount
```

Thread 1 sends a $\rightarrow$ b, thread 2 sends b $\rightarrow$ a, 100,000 times each.

DO

1. Run it. How do you know it is stuck?
2. Explain the cycle.
3. Fix it with a lock order.

Source: measured by staff on a 10-core laptop, CPython 3.14; your numbers will differ.

Exercise: solutions

Exercise	What happens	Fix
1. threads vs processes	threads $\approx 4\times$ one call (GIL)	processes for CPU-bound Python
2. word counts	get … set: updates lost	one lock around both, or per-thread counts
3. transfers	a $\rightarrow$ b and b $\rightarrow$ a: cycle	lock in a fixed order (by id)

In our pipeline

The best fix for a race is often **no sharing at all**: each worker keeps its own result and we **combine** them at the end — exactly what Spark and all-reduce do.

Key takeaways

- A **process** has its own address space; **threads** share their process's memory.
- **Concurrency** overlaps tasks in time; **parallelism** runs them at once on several cores.
- Shared writes **race**; **locks** fix races but serialize, and taken out of order they **deadlock**.
- Python's **GIL**: threads for waiting, **processes** for CPU-bound Python.
- **Amdahl**: $S(p) = 1 \,/\, ((1 - f) + f/p) \leq 1/(1 - f)$. Shrink the serial part.

Next time, and readings

When	What
Wednesday (this week)	D2 due, before the recess
Week 6, Lecture 1	Networking: latency, bandwidth, the $\alpha + \beta n$ cost of a message, sockets
Week 6, Lecture 2	Serialization, RPC, and datacenter networks; D3 checkpoint

READ BEFORE WEEK 6

Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*: ch. 4, 26, 28, 32.

Python docs: `threading`, `multiprocessing`, `concurrent.futures`.